Assignment 2

7 Points Possible

13/10/2026

Attempt 1

In Progress

NEXT UP: Submit Assignment

Add Comment

Unlimited Attempts Allowed

13/10/2026

Details

Assignment 2: Heap Go Solver with Minimax Search

Due: **Tuesday, Oct 13**, 23:55pm. Submit here on Canvas. The deadline is extended compared to the syllabus date since Oct 12 is a holiday.

Responsible TA for this assignment: Andrew (see teaching team page)

Resources

- Starter code: `assignment2.tgz` (https://canvas.ualberta.ca/courses/34644/files/9394069?wrap=1) (https://canvas.ualberta.ca/courses/34644/files/9394069/download?download_frd=1)
  - Decompress with `tar -xvzf assignment2.tgz`
  - The starter code contains an implementation of Heap Go and related text commands (a slightly adapted Assignment 1 solution), plus some commands for you to implement or modify for this Assignment 2.
- Review these as needed:
  - All Assignments - Rules and Policies (https://canvas.ualberta.ca/courses/34644/assignments/776003)
  - Rules of Heap Go (https://canvas.ualberta.ca/courses/34644/pages/rules-of-heap-go)
  - Assignment 1 (https://canvas.ualberta.ca/courses/34644/assignments/813356)
  - SSH connection guide (https://canvas.ualberta.ca/courses/34644/pages/ssh-connection-guide) - connect to UCOMM machines (required for testing your code)
  - Lecture slides (https://canvas.ualberta.ca/courses/34644/pages/lecture-slides) and sample code (https://github.com/ualberta-mueller-group/CMPUT455_code) for minimax search
  - Measuring Time in Python (https://canvas.ualberta.ca/courses/34644/pages/measuring-time)
- Assignment 2 preview slides (https://canvas.ualberta.ca/courses/34644/files/9196595?wrap=1) (https://canvas.ualberta.ca/courses/34644/files/9196595/download?download_frd=1) from class

Assignment 2 Specification

Write a Python3 program for solving Heap Go positions as win or loss. Who wins, if both players play perfectly? You solve this question efficiently by writing an optimised boolean minimax

search. Your solution is based on our `a2.py` starter code from `assignment2.tgz`.

Heap Go Restrictions for this Assignment

Compared to Assignment 1, the upper limits for number of heaps, and for number of tokens per heap, have been **increased to 20**. Restrictions for token values and komi are unchanged.

At the start of each game:

- The number of heaps is in the range **1 ≤ number of heaps ≤ 20**
- Each heap contains a number of tokens, in the range **1 ≤ tokens in heap ≤ 20**
- Each token (c, n) has color c, either 'b' or 'w', and integer value n, in the range 1 ≤ n ≤ 20
- The komi is an integer + 0.5 to avoid draws, in the range -100 < komi < +100

For this assignment, we will only test your `a2.py` program with valid input. You need to deal correctly with **time limits**. Other error handling will not be tested.

The **resource limits** for running and submitting your code are: 1 thread, 1 GB of memory at runtime, 1MB tgz file.

We will attempt to reply to all student questions, but you are expected to read about the assignment first. If the answer is already in the assignment guidelines, or in this spec, or linked from this spec, or in the discussion, then our reply might be short, as in "read the spec".

Getting Started

Download `assignment2.tgz` and expand it into a directory `assignment2`, which contains:

- Our starter code for assignment 2, `a2.py`. This implements a random player, and contains placeholders for commands that you need to implement/modify.
- A file `assignment2-public-tests.txt`, with test cases for your program. Also use this for your presubmission test. See details below.
- A program `a2test.py`. Use this code to test your solution with

```
python3 a2test.py a2.py assignment2-public-tests.txt
```

Goals of the Assignment

Implement the following functionality by adding or modifying existing text commands:

1. Implement a command

   ```
   timelimit seconds
   ```

   The argument `seconds` is an integer in the range `1 ≤ seconds ≤ 100`. This command sets the maximum time to use for all following `solve` commands, until it is changed by another `timelimit` command.

   Before the first `timelimit` command is given, the time limit is 1 second. The public test cases

show examples.

2. Implement a new command `solve` which takes no argument, and attempts to compute the winner of the current position, assuming perfect play by both.

   In case of successful solving, your text response should be in the format:

   ```
   winner [move]
   = 1
   ```

   In the response, `winner` is either `b` or `w`. `[move]` is an optional output explained below.

   In case of *time out*, when your solver cannot solve the game within the current time limit: Do not write a winner or move, just return a status:
   `= -1`
   Do not attempt to guess the result.
   Solving always starts with the current player (`self.game.toplay` in the sample code) going first.

   If `toplay` wins with best play by both, then also write a winning move: A legal move that you can play in the current position that achieves the win. Moves should be written as an integer `heap_number starting at 0,` as in Assignment 1, for example write `3` if taking from heap 3 is your winning move.

3. Change the `genmove` command in `a2.py` as follows: To generate a computer move, first call your solver with the current timelimit and try to solve the position.
   A. If your solver finds a win: return a winning move
   B. If your solver finds a loss or times out: return a random legal move (as in the starter code). Note that this is different from the `solve` command above - it allows you to always play a full game, since it generates a move even in those positions where your solver cannot solve.

Special Cases and Details

- The `solve` and `genmove` commands will **not** be called in case the game is over (all heaps are empty)
- Be aware that `toplay` is changed by several commands, as in Assignment 1, such as `play,` `toplay,` and more.
- If there are several winning moves, then write **any one** of them. Our test scripts will know the set of all winning moves in each case. If there are no winning moves - if the winner is the opponent of toplay - then do not write any move in your response, just write the winner `b` or `w`. See the examples of both scenarios (win and loss) in the public test cases.

Public Test Cases

Our test cases are roughly sorted by difficulty (as measured by our weakest sample solution) from

easier to harder. Some tests will be impossibly hard, and are designed to test your time management code. Tests fail for the starter code, since it does not implement the new commands. Public test cases only cover a small subset of possibilities. For full evaluation, we will use a more comprehensive set of private tests.

Other Text Commands

Other text commands from Assignment 1 are already implemented in the starter code. Do not break their functionality, since we will test your program by setting up test cases using the existing commands such as `heapgo,` `play, toplay`.

You are free to add to the implementations of these commands if you need it. For example, depending on your approach, you may need to do something extra in the play command. But do not break the existing functionality. We will have multiple test cases, set up with commands such as `heapgo` and `play`, within the same text file. Specifically, your solver should work for `toplay` being either Black or White.

Pre-submission Test and Submission

Follow the same general steps as in Assignment 1 to create your submission, test it on a lab machine, and create a log file. However, (of course) use your assignment2 directory, assignment2.tgz as submission file name, and test it with `assignment2-public-tests.txt`. Remember to include your current log file and `readme.txt` in your submission. The AI use form is recommended but optional.

Also, similar to Assignment 1:

- The submission must be a valid tgz file named `assignment2.tgz`
- Leave the name and location of your `a2.py` main file as in the starter code.
- Do not add directories or change the `assignment2` directory name.
- You may add extra python files within the same directory.

Hints and Details - Read them All

- Your solve command must stay within the time limit. For some ideas of how to do that in Python 3, see the link *Measuring Time in Python* in the *Resources* section above. Whatever you do, test that it works on the standard lab machines.
- **Please be considerate in your use of shared resources such as lab machines, especially close to the deadline.**
- No special error handling code is required in this assignment. We will only test with valid commands which have valid arguments.
- Set up your Assignment 2 Group, or submit as an individual, as desired. You need to set up an Assignment 2 group even if it has the same members as your Assignment 1 group.
- Being honest and returning "= -1" when your solver does not solve a position in time will lead to higher marks than randomly guessing the outcome.

- If you want to improve the efficiency of your solver, there is a large number of different approaches you can take. Some are discussed in our minimax lecture slides. We also have some hints and ideas in the Assignment 2 preview slides, but leave it up to you what kinds of improvements you try. In any case, make sure of the correctness of your solver. You will lose marks if your program just guesses the result, or does not compute it correctly because of bugs. Thorough testing is the key. Leave enough time for that.

Marking

There will be 5 marks + 2 possible bonus marks for this assignment.

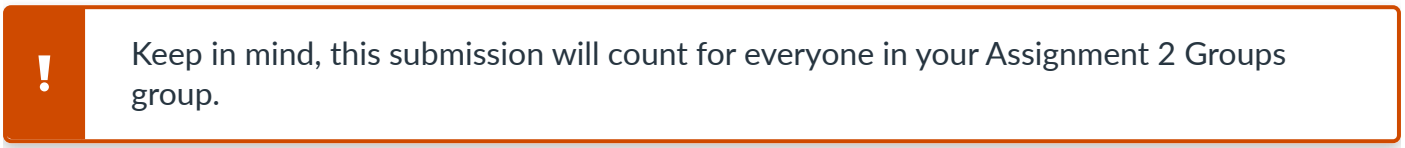
- 1 mark for your file `test.log` which shows the log of your correct test execution on **a standard Linux undergraduate machine**, and your `readme.txt` file with team information.
- 1 mark for passing our public test cases
- 1 mark for code quality. More details on page: all assignments - rules and policies
- 2 marks for passing our private test cases

Code that does not run, or just hardcodes answers to public test cases, will receive a mark of 0. If your code needs fixes, you need to submit a revised version **before the submission deadline**. TA will not attempt to fix your code under any circumstances. Use the discussion forum or consult the teaching team in case of questions.

Our tests contain a mix of easy, medium, hard, very hard and impossible cases. For the hardest test cases, your solver still needs to behave correctly (time out as) on those as well.

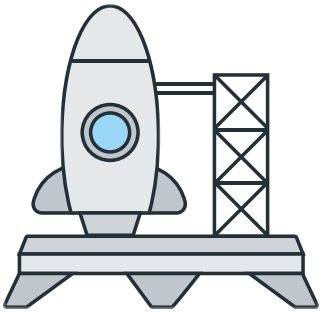
Bonus marks **may** be awarded at the discretion of the instructor and are expected to be rare - for exceptional solvers which are clearly above the other submissions.

Last modified: Sep 23, 2026 by Martin Müller



Choose a submission type





Drag a file here, or

Choose a file to upload

File permitted: TGZ

or

Canvas Files

Submit Assignment